VulnerableVault Protocol Audit Report

Version 1.0

alva-p.xyz

April 3, 2026

VulnerableVault Protocol Audit Report

alva-p

April 3, 2026

Prepared by: alva-p Lead Auditors: - Alvaro Pineda

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
  - Medium
  - Low
  - Informational

Protocol Summary

VulnerableVault is a simple token vault protocol deployed on the BattleChain testnet. It consists of two core contracts:

1. **MockToken** (`0xE6D85c2B5768F2819F1d8136784cADdbd8EA6547`): An ERC20 token with open minting and a transfer hook system that allows any address to register a callback contract invoked on incoming transfers.
2. **VulnerableVault** (`0xF3Ab51cd5cE66CCBf3e388eFBE80b56Aa571f71D`): A token vault that allows users to deposit and withdraw MockToken. Users deposit tokens via `deposit()` and retrieve their full balance via `withdrawAll()`.

The protocol is intentionally vulnerable as part of a BattleChain CTF / whitehat challenge. The primary vulnerability is a reentrancy attack enabled by a CEI (Checks-Effects-Interactions) violation in the vault's withdrawal function, combined with the token's transfer hook mechanism.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

- **Commit Hash:** d532f01 (branch: main)
- **Solidity Version:** 0.8.24
- **Framework:** Foundry
- **Network:** BattleChain Testnet
- **Static Analysis Tool:** Slither v0.10+ (19 findings across 101 detectors)

- **Dynamic Testing:** Foundry test suite (1 test, passing – reentrancy PoC confirmed)

Scope

```
./src/
  VulnerableVault.sol
  MockToken.sol
  Attacker.sol
```

Deployed Addresses:

Contract	Address
VulnerableVault	`0xF3Ab51cd5cE66CCBf3e388eFBE80b56Aa571f71D`
MockToken	`0xE6D85c2B5768F2819F1d8136784cADdbd8EA6547`
Attacker	`0x6753d455F282B35cA75649e4B5aEa69D79951674`

Roles

- **Depositor** – Any address that deposits MockToken into the VulnerableVault via `deposit()`.
- **Withdrawer** – Any depositor who calls `withdrawAll()` to retrieve their tokens.
- **Token Minter** – Any address (no access control on `mint()`).
- **Hook Registrant** – Any address that registers a transfer hook via `setTransferHook()`.

Executive Summary

The VulnerableVault protocol contains a **critical reentrancy vulnerability** that allows complete drainage of all vault funds in a single transaction. The attack is enabled by the combination of two design flaws: (1) a CEI violation in `VulnerableVault::withdrawAll()` where the token transfer occurs before the balance is zeroed, and (2) MockToken's unrestricted transfer hook system that provides the reentry vector.

Additionally, MockToken has no access control on its `mint()` function, allowing unlimited token creation by any address. Multiple unchecked ERC20 return values were also identified.

The exploit was verified via Foundry testing and **executed on-chain** on BattleChain testnet (Chain ID 627, Block 4403). 1,000 tokens were drained from the vault in a single transaction, with the attacker needing only a 100-token seed deposit.

Issues found

Severity	Number of issues found
High	1
Medium	3
Low	2
Info	4
Total	10

Findings

High

[H-1] Reentrancy in `VulnerableVault::withdrawAll()` allows complete vault drainage via MockToken transfer hook

Severity: High (Impact: High, Likelihood: High)

Location: `src/VulnerableVault.sol`, lines 40-51

Description:

The `withdrawAll()` function violates the Checks-Effects-Interactions (CEI) pattern. It performs the token transfer (Interaction) on line 45 **before** zeroing the caller's balance (Effect) on line 48:

```
1  function withdrawAll() external {
2      uint256 amount = balances[msg.sender];
3      require(amount > 0, "nothing to withdraw");
4
5      // INTERACTION before EFFECT
6      TOKEN.transfer(msg.sender, amount); // line 45 -- external call
7
8      // EFFECT after INTERACTION
9      balances[msg.sender] = 0;           // line 48 -- too late
10
11     emit Withdrawn(msg.sender, amount);
12 }
```

MockToken's `transfer()` function (line 41-50 of `MockToken.sol`) checks if the recipient has a registered transfer hook and calls `onTokenTransfer()` on that hook contract **after** the token

transfer completes. This callback provides the reentry vector: the attacker's hook calls `withdrawAll()` again while `balances[msg.sender]` still holds the original non-zero value.

The attack chain is:

1. Attacker deploys a contract implementing `ITransferHook` and registers it via `MockToken::setTransferHook()`.
2. Attacker mints seed tokens (open `mint()`) and deposits them into the vault.
3. Attacker calls `withdrawAll()`.
4. Vault calls `TOKEN.transfer(attacker, amount)` – tokens move.
5. MockToken detects the attacker's hook and calls `onTokenTransfer()`.
6. Inside `onTokenTransfer()`, the attacker calls `withdrawAll()` again. Since `balances[attacker]` has not been zeroed yet, the `require(amount > 0)` check passes.
7. Steps 4-6 repeat recursively until `TOKEN.balanceOf(vault)== 0`.
8. All the nested calls unwind, and `balances[attacker] = 0` executes once – after all funds are already gone.

Impact:

- **Total loss of all vault funds.** Every token deposited by every user is drained.
- The attacker only needs a small seed deposit (e.g., 100 tokens) to drain an arbitrarily large vault.
- Legitimate depositors' recorded balances become unwithdrawable (the vault has no tokens left but `balances[user]` is still non-zero).
- This was confirmed in the Foundry test: a 1,000-token vault was completely drained with a 100-token seed.

Proof of Concept:

The provided `Attacker.sol` contract and `ReentrancyTest.t.sol` test fully demonstrate the exploit. Running `forge test -vvv` confirms:

```
Vault balance before: 1000 tokens
--- Vault drained ---
Vault before:       1000 tokens
Vault after:        0 tokens
Bounty kept:        110 tokens
Returned to protocol: 990 tokens
```

Slither also flagged this as `reentrancy-no-eth` with cross-function reentrancy affecting `balances`, `deposit()`, `getBalance()`, and `withdrawAll()`.

On-Chain Execution (BattleChain Testnet – Chain ID 627):

The exploit was successfully executed on-chain on April 3, 2026:

Detail	Value
Attacker Contract	`0x6753d455F282B35cA75649e4B5aEa69D79951674`
Deploy TX	`0x798f6cdb7a6ee50c306c73b2fee91bf68dbcfd586`
Attack TX	`0x1ad7e14b36d74dd221c392fea35ad45377103c6a1`
Block	4403
Gas Used	1,052,810 (0.000107 ETH)
Vault Before	1,000 tokens
Vault After	0 tokens
Attacker Balance	1,100 tokens (1,000 drained + 100 seed)

Recommended Mitigation:

Apply the CEI pattern – zero the balance **before** making the external call. Additionally, add a reentrancy guard as defense in depth.

```
function withdrawAll() external nonReentrant {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

    // EFFECT first
    balances[msg.sender] = 0;
    emit Withdrawn(msg.sender, amount);

    // INTERACTION second
    TOKEN.transfer(msg.sender, amount);
}
```

Use OpenZeppelin's `ReentrancyGuard` for the `nonReentrant` modifier as a safety net. The CEI fix alone is sufficient to prevent this specific attack, but the guard protects against future modifications that might re-introduce interaction-before-effect patterns.

Medium

[M-1] `MockToken::mint()` has no access control – anyone can mint unlimited tokens

Severity: Medium (Impact: High, Likelihood: Medium)

Location: `src/MockToken.sol`, lines 29-31

Description:

The `mint()` function is completely unrestricted:

```
function mint(address to, uint256 amount) external {
    _mint(to, amount);
}
```

Any address can call `mint()` with an arbitrary `to` address and `amount`. While this is intentional for the BattleChain tutorial, in any production context this is a critical flaw. An attacker can mint unlimited tokens, deposit them into the vault, and dilute or manipulate the vault's accounting.

This also directly enables the reentrancy attack in [H-1] by allowing the attacker to create seed tokens at zero cost.

Impact:

- Unlimited token inflation.
- Attacker can create tokens to deposit into any protocol integrating with MockToken.
- Combined with the vault's reentrancy, this enables a zero-cost exploit (mint seeds, drain vault, keep everything).

Recommended Mitigation:

Restrict minting to an authorized role:

```
address public owner;

modifier onlyOwner() {
    require(msg.sender == owner, "not owner");
    _;
}

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

Alternatively, use OpenZeppelin's `AccessControl` or `Ownable` pattern.

[M-2] Unrestricted transfer hook registration enables arbitrary callback injection

Severity: Medium (Impact: High, Likelihood: Medium)

Location: `src/MockToken.sol`, lines 34-37

Description:

The `setTransferHook()` function allows **any address** to register an arbitrary contract as a callback hook:

```
function setTransferHook(address hook) external {
    transferHooks[msg.sender] = hook;
    emit TransferHookSet(msg.sender, hook);
}
```

This hook is then called inside `transfer()` (line 44-46) whenever tokens are sent to the registrant:

```
address hook = transferHooks[to];
if (hook != address(0)) {
    ITransferHook(hook).onTokenTransfer(msg.sender, amount);
}
```

This design converts every `transfer()` call into a potential external call to attacker-controlled code. Any protocol that calls `MockToken.transfer()` (such as VulnerableVault) is exposed to reentrancy or other callback-based attacks. The hook mechanism also only applies to `transfer()` but **not** to `transferFrom()`, creating inconsistent behavior.

Impact:

- Any contract calling `MockToken.transfer()` to send tokens to an address with a registered hook will execute arbitrary external code.
- This is the enabling mechanism for the reentrancy in [H-1].
- The inconsistency between `transfer()` (has hooks) and `transferFrom()` (no hooks) can lead to unexpected behavior.

Recommended Mitigation:

1. Remove the hook mechanism entirely if not needed.
2. If hooks are required, apply them consistently to both `transfer()` and `transferFrom()`.
3. Consider using ERC-777 or ERC-1363 standards which have established patterns for transfer callbacks with proper reentrancy protections.
4. At minimum, document the reentrancy risk prominently so integrating protocols are aware.

[M-3] Unchecked ERC20 return values in VulnerableVault and Attacker contracts

Severity: Medium (Impact: Medium, Likelihood: Medium)

Location: - `src/VulnerableVault.sol`, line 31 (`transferFrom`) and line 45 (`transfer`) - `src/Attacker.sol`, lines 82-83 (`transfer`)

Description:

Multiple ERC20 `transfer()` and `transferFrom()` calls do not check the boolean return value:

```
// VulnerableVault.sol
TOKEN.transferFrom(msg.sender, address(this), amount); // line 31 -- unchecked
TOKEN.transfer(msg.sender, amount);                    // line 45 -- unchecked

// Attacker.sol
TOKEN.transfer(RECOVERY_ADDRESS, toReturn);            // line 82 -- unchecked
TOKEN.transfer(OWNER, bounty);                         // line 83 -- unchecked
```

Per the ERC20 standard, `transfer()` and `transferFrom()` return `bool` to indicate success or failure. Some token implementations return **`false`** instead of reverting on failure. Ignoring these return values means the contract assumes the transfer succeeded when it may not have.

Slither flagged all four instances under the `unchecked-transfer` detector.

Impact:

- In `deposit()`: if `transferFrom` silently fails, the vault credits the user with tokens it never received, creating a debt.
- In `withdrawAll()`: if `transfer` silently fails, the user's balance is zeroed but they received no tokens – permanent loss.
- In the Attacker contract: bounty distribution could silently fail.

Recommended Mitigation:

Use OpenZeppelin's `SafeERC20` library:

```
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

using SafeERC20 for IERC20;

// Replace:
TOKEN.transfer(msg.sender, amount);
// With:
```

```
TOKEN.safeTransfer(msg.sender, amount);
```

Low

[L-1] `VulnerableVault::deposit()` has a benign reentrancy and event ordering issue

Severity: Low (Impact: Low, Likelihood: Low)

Location: `src/VulnerableVault.sol`, lines 30-34

Description:

The `deposit()` function calls `transferFrom()` before updating `balances` and emitting the event:

```
function deposit(uint256 amount) external {
    TOKEN.transferFrom(msg.sender, address(this), amount); // external call
    balances[msg.sender] += amount;                        // state update after
    emit Deposited(msg.sender, amount);                    // event after
}
```

While this is a CEI violation, it is benign in the current context because: 1. `transferFrom()` transfers tokens **to** the vault (not from it), so a reentrant call to `deposit()` would just deposit more tokens. 2. A reentrant call to `withdrawAll()` during deposit would withdraw the caller's existing balance, which is a legitimate operation.

However, the event ordering issue means that if MockToken's `transferFrom()` were modified to include hooks (currently only `transfer()` has hooks), events could be emitted in an incorrect order.

Slither flagged this as `reentrancy-benign` and `reentrancy-events`.

Impact:

- Currently no exploitable impact.
- Events may be emitted out of order if the token triggers callbacks during `transferFrom()`.
- Violates best practices and could become exploitable if the token contract is modified.

Recommended Mitigation:

Follow CEI even in deposit:

```
function deposit(uint256 amount) external {
```

```
    balances[msg.sender] += amount;
    emit Deposited(msg.sender, amount);
    TOKEN.transferFrom(msg.sender, address(this), amount);
}
```

Note: this changes semantics slightly – the balance is credited before tokens arrive. For production, use a pull-then-credit pattern with actual balance checks:

```
function deposit(uint256 amount) external {
    uint256 before = TOKEN.balanceOf(address(this));
    TOKEN.transferFrom(msg.sender, address(this), amount);
    uint256 received = TOKEN.balanceOf(address(this)) - before;
    balances[msg.sender] += received;
    emit Deposited(msg.sender, received);
}
```

[L-2] Missing zero-address validation in `Attacker` constructor for `_recoveryAddress`

Severity: Low (Impact: Medium, Likelihood: Low)

Location: `src/Attacker.sol`, line 38, 43

Description:

The `Attacker` constructor does not validate that `_recoveryAddress` is not `address(0)`:

```
constructor(
    address _vault,
    address _token,
    address _recoveryAddress, // no zero-check
    uint256 _bountyBps
) {
    // ...
    RECOVERY_ADDRESS = _recoveryAddress;
    // ...
}
```

If `_recoveryAddress` is set to `address(0)`, the 90% recovery transfer on line 82 would send tokens to the zero address, effectively burning them.

Slither flagged this under `missing-zero-check`.

Impact:

- If deployed with `_recoveryAddress = address(0)`, 90% of recovered funds would be burned.
- Since `RECOVERY_ADDRESS` is immutable, this cannot be corrected after deployment.

Recommended Mitigation:

```
require(_recoveryAddress != address(0), "zero recovery address");
```

Informational

[I-1] No partial withdrawal function in VulnerableVault

Location: `src/VulnerableVault.sol`

Description:

The vault only provides `withdrawAll()` with no option to withdraw a specific amount. This forces users into an all-or-nothing withdrawal pattern, which reduces flexibility and could lead to unnecessary gas costs or unfavorable timing for large positions.

Recommended Mitigation:

Add a `withdraw(uint256 amount)` function:

```
function withdraw(uint256 amount) external nonReentrant {
    require(balances[msg.sender] >= amount, "insufficient balance");
    balances[msg.sender] -= amount;
    emit Withdrawn(msg.sender, amount);
    TOKEN.safeTransfer(msg.sender, amount);
}
```

[I-2] Missing `nonReentrant` guard – no reentrancy protection mechanism exists

Location: `src/VulnerableVault.sol`

Description:

The VulnerableVault contract does not inherit from `ReentrancyGuard` and has no reentrancy lock mechanism. Even if the CEI violation in `withdrawAll()` were fixed, there would be no defense-in-depth protection against future code changes that might reintroduce reentrancy vectors.

Recommended Mitigation:

Inherit from OpenZeppelin's `ReentrancyGuard` and apply `nonReentrant` to all state-changing functions:

```
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
    ReentrancyGuard.sol";

contract VulnerableVault is ReentrancyGuard {
```

```
    // ...
    function deposit(uint256 amount) external nonReentrant { ... }
    function withdrawAll() external nonReentrant { ... }
}
```

[I-3] `Attacker` contract does not formally inherit `ITransferHook` interface

Location: `src/Attacker.sol`

Description:

The `Attacker` contract implements the `onTokenTransfer(address, uint256)` function but does not formally inherit from the `ITransferHook` interface defined in `MockToken.sol`. While this works because Solidity uses function selectors for dispatch, formally inheriting the interface provides compile-time guarantees that the function signature matches.

Slither flagged this under `missing-inheritance`.

Recommended Mitigation:

```
import {ITransferHook} from "./MockToken.sol";

contract Attacker is ITransferHook {
    // ...
}
```

[I-4] No ERC4626 vault inflation attack protection

Location: `src/VulnerableVault.sol`

Description:

While VulnerableVault is a custom vault (not ERC4626), it uses a similar deposit/withdraw pattern with internal balance tracking. The vault does not implement any protection against the first-depositor inflation attack, where an attacker can manipulate the share-to-asset ratio. In this specific vault the accounting is 1:1 (tokens deposited = balance credited), so the classic inflation attack does not directly apply. However, this is noted for awareness if the vault is extended to use a shares-based model.

Recommended Mitigation:

If converting to a shares-based vault, implement virtual shares/assets offset (e.g., `_decimalsOffset()` in OpenZeppelin's ERC4626) or require a minimum initial deposit.

Recommendations Summary

Listed in order of priority:

1. **[CRITICAL] Fix the CEI violation in `withdrawAll()`** – Zero `balances[msg.sender]` before calling `TOKEN.transfer()`. This is the single most important fix.
2. **HIGH Add `ReentrancyGuard`** – Apply `nonReentrant` modifier to `deposit()` and `withdrawAll()` as defense in depth.
3. **HIGH Use `SafeERC20`** – Replace all raw `transfer()`/`transferFrom()` calls with `safeTransfer()`/`safeTransferFrom()`.
4. **MEDIUM Restrict `MockToken::mint()`** – Add access control (Ownable or AccessControl).
5. **MEDIUM Redesign or remove the transfer hook system** – Or at minimum apply hooks consistently to both `transfer()` and `transferFrom()`, and document the reentrancy risk.
6. **LOW Add zero-address checks** – Validate constructor parameters.
7. **LOW Fix CEI ordering in `deposit()`** – Update balance before external call, or use balance-difference pattern.
8. **[INFO] Add partial withdrawal** – Implement `withdraw(uint256 amount)`.
9. **[INFO] Inherit interfaces formally** – `Attacker` should inherit `ITransferHook`.

Appendix A: Slither Results Summary

Slither analyzed 13 contracts with 101 detectors and produced 19 results. Key findings mapped to this report:

Slither Detector	Instances	Report Finding
`reentrancy-no-eth`	1	[H-1]
`unchecked-transfer`	4	[M-3]
`reentrancy-benign`	1	[L-1]
`reentrancy-events`	2	[L-1]
`missing-zero-check`	1	[L-2]
`unused-return`	1	[M-3]
`missing-inheritance`	3	[I-3]
`naming-convention`	6	Not reported (style)

Appendix B: Test Results

```
Ran 1 test for test/ReentrancyTest.t.sol:ReentrancyTest
[PASS] test_reentrancyDrainsVault() (gas: 2749634)

Vault balance before: 1000 tokens
Vault after:          0 tokens
Bounty kept:          110 tokens (10%)
Returned to protocol: 990 tokens (90%)
```

The test confirms that the reentrancy attack fully drains the vault. The attacker deposited 100 tokens as seed and extracted all 1,100 tokens (1,000 vault + 100 seed), keeping 110 as bounty and returning 990 to the recovery address.

Appendix C: On-Chain Execution Proof

The exploit was executed on BattleChain testnet on April 3, 2026:

- **Deploy TX:** `0x798f6cdb7a6ee50c306c73b2fee91bf68dbcfd586c4a6a83ce00954e8f6c11f8`
- **Attack TX:** `0x1ad7e14b36d74dd221c392fea35ad45377103c6a185704d8ceeb7436ff466d39`
- **Attacker Contract:** `0x6753d455F282B35cA75649e4B5aEa69D79951674`
- **Block:** 4403
- **Chain ID:** 627
- **Total Gas:** 1,052,810 (0.000107 ETH)
- **Result:** Vault fully drained – 0 tokens remaining